

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №5
«Процедуры, функции, триггеры в PostgreSQL»**

Обучающийся Заботкина Марина Александровна
Факультет прикладной информатики
Группа К3239
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии 2024
Преподаватель Говорова Марина Михайловна

Санкт-Петербург

2025/2026

Цель работы

Овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL

Практическое задание

1. Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2). Допустимо использование IN/INOUT параметров. Допустимо создать авторские процедуры. (3 балла)
2. Создать триггеры для индивидуальной БД согласно варианту:
Вариант 2.1. 7 триггеров по ИЗ.

Выполнение

1. Хранимые процедуры

- 1) Снижение цены на заданный процент для товаров, у которых срок пребывания на складе превысил заданный норматив.

```
CREATE OR REPLACE PROCEDURE wholesale_base.reduce_price_old_batches( IN
p_days_norm INT, IN p_discount_percent NUMERIC )

LANGUAGE plpgsql AS $$

BEGIN    UPDATE    wholesale_base.batch    SET    purchase_price    =
wholesale_base.batch.purchase_price * (1.0- (p_discount_percent / 100.0)) FROM
wholesale_base.supply    WHERE    wholesale_base.batch.id_supply    =
wholesale_base.supply.id_supply    AND    (CURRENT_DATE    -
wholesale_base.supply.date_supply) > p_days_norm;

END; $$;
```

Цена товара до выполнения процедуры:

Query

Query History

1

SELECT b.id_batch, s.date_supply, b.purchase_price

2

FROM wholesale_base.batch b

3

JOIN wholesale_base.supply s **ON** b.id_supply = s.id_supply

4

WHERE b.id_batch = 1;

5

Data Output

Messages

Notifications

</

Выполнение процедуры:

Query	Query History
1	CALL wholesale_base.reduce_price_old_batches(5, 10.0);
2	
Data Output	Messages Notifications
CALL	
Query returned successfully in 74 msec.	

Цена товара после выполнения процедуры:

Query

Query History

1

SELECT b.id_batch, s.date_supply, b.purchase_price

2

FROM wholesale_base.batch b

3

JOIN wholesale_base.supply s ON b.id_supply = s.id_supply

4

WHERE b.id_batch = 1;

5

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 1

	id_batch bigint	date_supply date	purchase_price numeric (10,2)
1	1	2024-10-01	76500.00

2) Расчет стоимости всех партий товаров, проданных за прошедшие сутки.

```
CREATE OR REPLACE PROCEDURE wholesale_base.calc_yesterday_revenue( INOUT  
p_total_revenue NUMERIC DEFAULT 0 )  
  
LANGUAGE plpgsql AS $$
```

```
BEGIN SELECT COALESCE(SUM(wholesale_base.order_content.sell_price *
wholesale_base.order_content.counts), 0) INTO p_total_revenue FROM
wholesale_base.order_content JOIN wholesale_base."order" ON
wholesale_base.order_content.id_order = wholesale_base."order".id_order
WHERE wholesale_base."order".export_date = CURRENT_DATE - INTERVAL '1 day';

END; $$;
```

Все партии товаров за прошедшие сутки:

Query Query History	
1	SELECT wholesale_base."order".export_date, wholesale_base.order_content.counts,
2	wholesale_base.order_content.sell_price,
3	(wholesale_base.order_content.counts * wholesale_base.order_content.sell_price) AS "total"
4	FROM wholesale_base."order" JOIN wholesale_base.order_content
5	ON wholesale_base."order".id_order = wholesale_base.order_content.id_order
6	WHERE wholesale_base."order".export_date = CURRENT_DATE - INTERVAL '1 day';

Data Output Messages Notifications	
SQL Showing rows: 1 to 4 Page No: 1 of 1	
export_date	counts
2026-05-10	10
2026-05-10	2
2026-05-10	1
2026-05-10	10

Вызов процедуры:

Query Query History	
1	CALL wholesale_base.calc_yesterday_revenue(0);

Data Output Messages Notifications	
SQL Showing rows: 1 to 1 Page	
p_total_revenue	
200000.00	

3) Расчет премии сотрудника при выполнении плана продаж

```
CREATE OR REPLACE PROCEDURE wholesale_base.calculate_worker_bonus( IN
p_id_worker BIGINT, IN p_quota NUMERIC, IN p_bonus_percent NUMERIC, INOUT
p_bonus_amount NUMERIC DEFAULT 0 )
LANGUAGE plpgsql AS $$ DECLARE v_total_sales NUMERIC := 0;
BEGIN SELECT COALESCE(SUM(oc.sell_price * oc.counts), 0) INTO v_total_sales
FROM wholesale_base."order" o JOIN wholesale_base.order_content oc ON
o.id_order = oc.id_order WHERE o.id_worker = p_id_worker; IF v_total_sales >=
```

```
p_quota p_bonus_amount := v_total_sales * (p_bonus_percent / 100.0); ELSE
p_bonus_amount := 0;
END IF;
END; $$;
```

Все продажи сотрудника с id = 1:

Query Query History

```
1 SELECT wholesale_base."order".id_order, wholesale_base."order".export_date,
2 wholesale_base.order_content.counts, wholesale_base.order_content.sell_price,
3 (wholesale_base.order_content.counts * wholesale_base.order_content.sell_price) AS "total"
4 FROM wholesale_base."order" JOIN wholesale_base.order_content
5 ON wholesale_base."order".id_order = wholesale_base.order_content.id_order
6 WHERE wholesale_base."order".id_worker = 1;
```

Data Output Messages Notifications

SQL

Showing rows: 1 to 13

Page No:

	id_order bigint	export_date date	counts integer	sell_price numeric (10,2)	total numeric
1	2	2024-10-12	1	47250.00	47250.00
2	3	[null]	1	26250.00	26250.00
3	5	[null]	50	840.00	42000.00
4	6	[null]	5	472.50	2362.50
5	7	[null]	2	57750.00	115500.00
6	8	[null]	20	367.50	7350.00
7	9	[null]	10	945.00	9450.00
8	10	[null]	30	126.00	3780.00
9	1	[null]	2	84787.50	169575.00
10	4	[null]	10	1496.25	14962.50
11	31	[null]	4	84787.50	339150.00
12	35	[null]	4	54862.50	219450.00
13	37	2026-05-10	10	5000.00	50000.00

Вызов процедуры (если сотрудник выполнил план):

Query Query History

```
1 CALL wholesale_base.calculate_worker_bonus(1, 1000000.00, 10.0, 0);
```

Data Output Messages Notifications

SQL

Showing rows: 1

	p_bonus_amount numeric
1	104708.00

Вызов процедуры (если сотрудник не выполнил план):

Query Query History

```
1 CALL wholesale_base.calculate_worker_bonus(1, 5000000.00, 10.0, 0);
```

Data Output Messages Notifications

Showing rows: 1 to 1

	p_bonus_amount numeric	
1	0	

2. Триггеры

1) Дата отгрузки не может быть раньше даты оформления заказа

```
CREATE OR REPLACE FUNCTION wholesale_base.fn_check_order_dates() RETURNS  
TRIGGER AS $$
```

```
BEGIN IF NEW.export_date < NEW.order_date THEN RAISE EXCEPTION 'Ошибка:  
Дата отгрузки (%) не может быть раньше даты заказа (%)', NEW.export_date,  
NEW.order_date;
```

```
END IF;
```

```
RETURN NEW;
```

```
END; $$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_check_order_dates BEFORE INSERT OR UPDATE ON  
wholesale_base."order" FOR EACH ROW EXECUTE PROCEDURE  
wholesale_base.fn_check_order_dates();
```

Результат неверной вставки данных:

Query Query History

```
1 INSERT INTO wholesale_base."order" (order_date, export_date, id_worker, id_warehouse, status, id_customer)  
2 VALUES ('2024-10-20', '2024-10-10', 1, 1, 'новый', 1);
```

Data Output Messages Notifications

ERROR: Ошибка: Дата отгрузки (2024-10-10) не может быть раньше даты заказа (2024-10-20)
CONTEXT: функция PL/pgSQL wholesale_base.fn_check_order_dates(), строка 4, оператор RAISE

ОШИБКА: Ошибка: Дата отгрузки (2024-10-10) не может быть раньше даты заказа (2024-10-20)
SQL state: P0001

2) Запрет изменения заказа, если он уже отгружен

```
CREATE OR REPLACE FUNCTION wholesale_base.fn_lock_shipped_order()
RETURNS TRIGGER AS $$

BEGIN IF OLD.status = 'отгружен' THEN RAISE EXCEPTION 'Редактирование
запрещено: заказ уже отгружен покупателю.';

END IF;

RETURN NEW;

END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_lock_shipped_order BEFORE UPDATE ON
wholesale_base."order" FOR EACH ROW EXECUTE PROCEDURE
wholesale_base.fn_lock_shipped_order();
```

Результат неверной вставки данных:

Query Query History

1 UPDATE wholesale_base."order" SET id_warehouse = 3 WHERE status = 'отгружен';

Data Output Messages Notifications

ERROR: Редактирование запрещено: заказ уже отгружен покупателю.
CONTEXT: функция PL/pgSQL wholesale_base.fn_lock_shipped_order(), строка 4, оператор RAISE

ОШИБКА: Редактирование запрещено: заказ уже отгружен покупателю.
SQL state: P0001

3) Автоматическая наценка (доход базы не менее 5% от стоимости закупки)

```
CREATE OR REPLACE FUNCTION wholesale_base.fn_enforce_5_percent_profit()
RETURNS TRIGGER AS $$

DECLARE v_purchase_price NUMERIC; v_min_sell_price NUMERIC;

BEGIN

SELECT purchase_price INTO v_purchase_price

FROM wholesale_base.batch WHERE id_batch = NEW.id_batch;-- v_min_sell_price
:= v_purchase_price * 1.05 IF NEW.sell_price IS NULL OR NEW.sell_price <
v_min_sell_price THEN NEW.sell_price := v_min_sell_price;

END IF;

RETURN NEW;

END; $$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_enforce_5_percent_profit BEFORE INSERT OR UPDATE ON
wholesale_base.order_content FOR EACH ROW EXECUTE PROCEDURE
wholesale_base.fn_enforce_5_percent_profit();
```

Вставляем данные в заказ с продажной ценой товара, которая ниже закупочной:

Query
Query History

```
1 INSERT INTO wholesale_base.order_content (id_order, id_batch, counts, sell_price)
2 VALUES (1, 25, 1, 1.00);
3
```

Data Output
Messages
Notifications

INSERT 0 1

Query returned successfully in 49 msec.

Результат неверной вставки данных (продажная цена автоматически вставляется как закупочная + 5%, заменяя неверную цену, вставленную вручную):

Query
Query History

```
1 SELECT wholesale_base.order_content.id_batch, wholesale_base.batch.purchase_price,
2 wholesale_base.order_content.sell_price,
3 ROUND(((wholesale_base.order_content.sell_price / wholesale_base.batch.purchase_price) - 1) * 100, 2)
4 FROM wholesale_base.order_content
5 JOIN wholesale_base.batch ON wholesale_base.order_content.id_batch = wholesale_base.batch.id_batch
6 WHERE wholesale_base.order_content.id_batch = 25;
```

Data Output
Messages
Notifications

Showing rows: 1 to 1
Page No: 1

	id_batch bigint	purchase_price numeric (10,2)	sell_price numeric (10,2)	round numeric
1	25	1000.00	1050.00	5.00

4) Автоматическое уменьшение остатка на складе при добавлении товара в заказ

```
CREATE OR REPLACE FUNCTION wholesale_base.fn_reduce_stock() RETURNS
TRIGGER AS $$
```

```
BEGIN UPDATE wholesale_base.stock SET count_of_stock = count_of_stock -
NEW.counts WHERE id_batch = NEW.id_batch;
```

```
RETURN NEW;
```

```
END; $$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_reduce_stock AFTER INSERT ON
wholesale_base.order_content FOR EACH ROW EXECUTE PROCEDURE
wholesale_base.fn_reduce_stock();
```

Смотрим остаток товара на складе:

Query Query History

```
1 SELECT id_stock, id_batch, count_of_stock
2 FROM wholesale_base.stock
3 WHERE id_batch = 2;
4
```

Data Output Messages Notifications

	id_stock [PK] bigint	id_batch bigint	count_of_stock integer
1	2	2	20

Вставляем данные в заказ, забирая из этой партии 10 товаров:

Query Query History

```
1 INSERT INTO wholesale_base.order_content (id_order, id_batch, counts, sell_price)
2 VALUES (1, 2, 10, 50000.00);
3
```

Data Output Messages Notifications

INSERT 0 1

Query returned successfully in 62 msec.

На складе количество этого товара уменьшилось на 10:

Query Query History

```
1 SELECT id_stock, id_batch, count_of_stock
2 FROM wholesale_base.stock
3 WHERE id_batch = 2;
```

Data Output Messages Notifications

	id_stock [PK] bigint	id_batch bigint	count_of_stock integer
1	2	2	10

5) Автоматическое удаление записи со склада, если товар закончился

```
CREATE OR REPLACE FUNCTION wholesale_base.fn_clean_empty_stock() RETURNS
TRIGGER AS $$

BEGIN IF NEW.count_of_stock <= 0 THEN DELETE FROM wholesale_base.stock
WHERE id_stock = NEW.id_stock;

END IF;

RETURN NEW;

END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_clean_empty_stock AFTER UPDATE ON wholesale_base.stock
FOR EACH ROW EXECUTE PROCEDURE wholesale_base.fn_clean_empty_stock();
```

Смотрим, что на складе под id = 3 есть партия товаров:

Query Query History

```
1 SELECT id_stock, id_batch, count_of_stock
2 FROM wholesale_base.stock
3 WHERE id_stock = 3;
```

Data Output Messages Notifications

	id_stock [PK] bigint	id_batch bigint	count_of_stock integer
1	3	3	10

Обнуляем количество товаров в этой партии:

Query Query History

```
1 UPDATE wholesale_base.stock SET count_of_stock = 0 WHERE id_stock = 3;
```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 111 msec.

Теперь под этим id нет партии на складе:

Query Query History

1 SELECT * FROM wholesale_base.stock WHERE id_stock = 3;

Data Output Messages Notifications

SQL

id_stock [PK] bigint count_of_stock integer id_warehouse bigint id_batch bigint

6) Запрет удаления клиента, если у него есть заказы

```
CREATE OR REPLACE FUNCTION wholesale_base.fn_prevent_customer_deletion()
RETURNS TRIGGER AS $$

BEGIN

IF EXISTS (SELECT 1 FROM wholesale_base."order" WHERE id_customer =
OLD.id_customer) THEN RAISE EXCEPTION 'Удаление невозможно: у клиента (ID:
%) есть история заказов.', OLD.id_customer;

END IF;

RETURN OLD;

END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_customer_deletion BEFORE DELETE ON
wholesale_base.customer FOR EACH ROW EXECUTE PROCEDURE
wholesale_base.fn_prevent_customer_deletion();
```

Смотрим, что у клиента с id = 1 есть заказы в базе:

Query Query History

1 SELECT id_order, order_date, status
2 FROM wholesale_base."order"
3 WHERE id_customer = 1;

Data Output Messages Notifications

Show

id_order [PK] bigint order_date date status character varying (20)

1	1	2024-10-10	оплачен
2	31	2024-11-01	оплачен
3	32	2024-11-03	новый
4	37	2026-05-10	отгружен

Пытаемся удалить клиента из базы:

Query	Query History
1	<code>DELETE FROM wholesale_base.customer WHERE id_customer = 1;</code>
Data Output	Messages
ERROR: Удаление невозможно: у клиента (ID: 1) есть история заказов. CONTEXT: функция PL/pgSQL wholesale_base.fn_prevent_customer_deletion(), строка 4, оператор RAISE ОШИБКА: Удаление невозможно: у клиента (ID: 1) есть история заказов. SQL state: P0001	

7) Автоматический возврат товара на склад при отмене заказа

```
CREATE OR REPLACE FUNCTION wholesale_base.fn_return_stock_on_cancel()
RETURNS TRIGGER AS $$

BEGIN IF NEW.status = 'отменен' AND OLD.status != 'отменен' THEN UPDATE
wholesale_base.stock SET count_of_stock = wholesale_base.stock.count_of_stock
+ wholesale_base.order_content.counts FROM wholesale_base.order_content
WHERE wholesale_base.order_content.id_order = NEW.id_order AND
wholesale_base.stock.id_batch = wholesale_base.order_content.id_batch; END IF;

RETURN NEW;

END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_return_stock_on_cancel AFTER UPDATE ON
wholesale_base."order" FOR EACH ROW EXECUTE PROCEDURE
wholesale_base.fn_return_stock_on_cancel();
```

Добавляем в заказ 5 товаров из партии:

Query	Query History
1	<code>INSERT INTO wholesale_base.order_content (id_order, id_batch, counts, sell_price)</code>
2	<code>VALUES (5, 4, 5, 2000.00);</code>
Data Output	Messages
INSERT 0 1 Query returned successfully in 66 msec.	

На складе осталось 95 штук:

Query Query History

```
1 SELECT id_batch, count_of_stock FROM wholesale_base.stock WHERE id_batch = 4;
```

Data Output Messages Notifications

Showing rows: 1 to 1

	id_batch bigint	count_of_stock integer
1	4	95

Меняем статус заказа на 'отменён':

Query Query History

```
1 UPDATE wholesale_base."order" SET status = 'отменен' WHERE id_order = 5;
```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 77 msec.

На склад вернулось 5 штук товара:

Query Query History

```
1 SELECT id_batch, count_of_stock FROM wholesale_base.stock WHERE id_batch = 4;
```

Data Output Messages Notifications

Showing rows: 1 to 1

	id_batch bigint	count_of_stock integer
1	4	100

3. Выводы

В ходе выполнения лабораторной работы были закреплены практические навыки программирования серверной части баз данных PostgreSQL с использованием хранимых процедур и триггеров.

Были разработаны процедуры для перерасчета стоимости устаревших партий товаров, автоматизированного расчета суточной выручки с использованием выходных параметров и алгоритмического вычисления премиальных выплат сотрудникам на основе выполнения плановых показателей. Это позволило значительно упростить

взаимодействие клиентского приложения с базой данных, сократив количество выполняемых SQL-запросов.

Созданный комплекс из семи триггеров позволил автоматизировать проверку бизнес-правил. Разработанные триггеры обеспечивают контроль актуальности данных (запрет изменения статуса уже отгруженных заказов и предотвращение логических ошибок), автоматизацию складского учета (товар автоматически списывается при добавлении в состав заказа и возвращается на склад при его отмене), защиту целостности ссылок (запрет на удаление записей клиентов с существующей историей заказов). Внедрен механизм автоматической корректировки цен продажи (обеспечение минимальной наценки в 5%), что исключает человеческий фактор при ценообразовании.

Бизнес-правила соблюдаются автоматически, независимо от действий пользователя в приложении, что позволяет базе данных работать корректно.